

关同步

```
ios::sync_with_stdio(0);
cin.tie(0);
```

快读

```
inline int read()
{
    char ch;
    int sign=1;
    while((ch=getchar())<'0' || ch>'9')
    if(ch=='-') sign=-1;
    int res=ch-48;
    while((ch=getchar())>='0' && ch<='9')
    res=res*10+ch-48;
    return res*sign;
}
```

基础算法

高精度

- 高精度加法

```
int a[202],b[202],c[202],x=0,lena,lenb,lenc;
memset(a,0,sizeof(a));
memset(b,0,sizeof(b));
memset(c,0,sizeof(c));
char a1[2000],b1[2000];
gets(a1);gets(b1);
lena=strlen(a1);
lenb=strlen(b1);
for(int i=0;i<=lena-1;i++) a[lena-i]=a1[i]-'0';
for(int j=0;j<=lenb-1;j++) b[lenb-j]=b1[j]-'0';
lenc=1;
while(lenc<=lena || lenc<=lenb)
{
    c[lenc]=a[lenc]+b[lenc]+x;
    x=c[lenc]/10;
    c[lenc]%=10;
    lenc++;
}
c[lenc]=x;
while(c[lenc]==0) lenc--;
for(int i=lenc;i>=1;i--) cout<<c[i];>=1;i--) cout<<c[i];
```

- 高精度减法

```
int a[202],b[202],c[202],la,lb,lc,i;
char a1[2000],b1[2000],n[2000];
memset(a,0,sizeof(a));
```

```

memset(b,0,sizeof(b));
memset(c,0,sizeof(c));
gets(a1);gets(b1);
if(strlen(a1)<strlen(b1)|| (strlen(a1)==strlen(b1)&&strcmp(a1,b1)<0))
{
    strcpy(n,a1);
    strcpy(a1,b1);
    strcpy(b1,n);
    cout<<"- ";
}
la=strlen(a1);
lb=strlen(b1);
for(i=0;i<=la-1;i++) a[la-i]=a1[i]-'0';
for(i=0;i<=lb-1;i++) b[lb-i]=b1[i]-'0';
i=1;
while(i<=la||i<=lb)
{
    if(a[i]<b[i]) {
        a[i]+=10;
        a[i+1]--;
    }
    c[i]=a[i]-b[i];
    i++;
}
lc=i;
while((c[lc]==0)&&(lc>1)) lc--;
for(i=lc;i>=1;i--) cout<<c[i];

```

- 高精度乘法

```

int a[101],b[101],c[10001],la,lb,lc,i,j,x;
memset(a,0,sizeof(a));
memset(b,0,sizeof(b));
memset(c,0,sizeof(c));
char a1[101],b1[101];
scanf("%s",a1);
scanf("%s",b1);
la=strlen(a1);
lb=strlen(b1);
for(i=0;i<=la-1;i++) a[la-i]=a1[i]-'0';
for(i=0;i<=lb-1;i++) b[lb-i]=b1[i]-'0';
for(i=1;i<=la;i++)
{
    x=0;
    for(j=1;j<=lb;j++)
    {
        c[i+j-1]=a[i]*b[j]+x+c[i+j-1];
        x=c[i+j-1]/10;
        c[i+j-1]%=10;
    }
    c[i+lb]=x;
}
lc=la+lb;
while((c[lc]==0)&&(lc>1)) lc--;
for(i=lc;i>=1;i--) cout<<c[i];

```

排序

- 快速排序

```
void qsort(int l,int r)
{
    int mid=a[(l+r)/2];
    int i=l,j=r;
    do
    {
        while(a[i]<mid) i++;
        while(a[j]>mid) j--;
        if(i<=j)
        {
            swap(a[i],a[j]);
            i++;j--;
        }
    }while(i<=j);
    if(l<j) qsort(l,j);
    if(i<r) qsort(i,r);
}
```

- 归并+求逆序对

```
void msort(int s,int t)
{
    if(s==t) return ;
    int mid=(s+t)/2;
    msort(s,mid);
    msort(mid+1,t);
    int i=s,j=mid+1,k=s;
    while(i<=mid&& j<=t)
    {
        if(a[i]<=a[j])
        {
            r[k]=a[i];
            k++;i++;
        }
        else
        {
            r[k]=a[j];
            k++;j++;
            ans+=mid-i+1;
        }
    }
    while(i<=mid)
    {
        r[k]=a[i];k++;i++;
    }
    while(j<=t)
    {
        r[k]=a[j];
        k++;j++;
    }
}
```

```

    }
    for(int i=s;i<=t;i++) a[i]=r[i];
}

```

二分

- 手动二分

//寻找最后一个满足条件（最大值）

```

int find()
{
    int l=1,r=m,ans=1;
    while(l<=r)
    {
        int mid=l+(r-l)/2;
        if(check(mid))
        {
            ans=mid;
            l=mid+1;
        }
        else r=mid-1;
    }
    return r;
}

```

//寻找第一个满足条件的（最小值）

```

auto find_first=[&](int x)
{
    int l=1,r=m,ans=m;
    while(l<=r)
    {
        int mid=l+(r-l)/2;
        if (check(mid)>=x)
        {
            ans=mid;
            r=mid-1;
        }
        else l=mid+1;
    }
    return ans;
};

```

- stl二分

```

//vector
auto lb = lower_bound(vec.begin(), vec.end(), 3);
auto ub = upper_bound(vec.begin(), vec.end(), 3);
cout << "lower_bound index: " << lb - vec.begin() << '\n';
cout << "upper_bound index: " << ub - vec.begin() << '\n';
//静态数组
auto lb=lower_bound(arr+1,arr+n+1,3);
auto ub=upper_bound(arr+1,arr+n+1,3);
cout<<"lower_bound index: "<<(lb-arr)<<'\n';
cout<<"upper_bound index: "<<(ub-arr)<<'\n';

```

- 三分

```
while (r - l > eps)
{
    mid = (l + r) / 2;
    lmid = mid - eps;
    rmid = mid + eps;
    if (f(lmid) < f(rmid))
        r = mid;
    else
        l = mid;
}
return f(l);
```

- 标准三分

```
//迭代
for(int i=0;i<100;i++)
{
    double lmid=l+(r-l)/3.0;
    double rmid=r-(r-l)/3.0;
    if(f(lmid)<f(rmid)) r=rmid;
    else l=lmid;
}
double ans=f((l+r)/2.0);
if(ans<0) ans=0;
return sqrt(ans);

//循环
while (r - l > eps) {
    double lmid = l + (r - l) / 3.0;
    double rmid = r - (r - l) / 3.0;
    if (f(lmid) < f(rmid)) r = rmid;    // 极小值
    else l = lmid;
}
return f((l + r) / 2.0);
```

STL

string

`s.insert(pos, args)` 在 `pos` 之前插入 `args` 指定的字符

`s.erase(pos, len)` 删除从 `pos` 开始的 `len` 个字符。如果 `len` 省略，则删除 `pos` 开始的后面所有字符。返回一个指向 `s` 的引用。

`s.assign(args)` 将 `s` 中的字符替换为 `args` 指定的字符。返回一个指向 `s` 的引用。

`s.append(args)` 将 `args` 追加到 `s`。返回一个指向 `s` 的引用。`args` 必须是双引号字符串

`s.replace(range, args)` 将 `s` 中范围为 `range` 内的字符替换为 `args` 指定的字符

`s.find(args)` 查找 `s` 中 `args` 第一次出现的位置

`s.rfind(args)` 查找 `s` 中 `args` 最后一次出现的位置

`to_string(val)` 将数值 `val` 转换为 `string` 并返回。`val` 可以是任何算术类型（`int`、浮点型等）

`stoi(s) / atoi(c)` 字符串/字符 转换为整数并返回

`stof(s) / atof(s)` 字符串/字符 转换为浮点数并返回

`s.substr(pos, n)` 从索引 `pos` 开始，提取连续的 `n` 个字符，包括 `pos` 位置的字符

```
reverse(s2.begin(), s2.end()) 反转 string 定义的字符串 s2
```

vector

```
vector<int> v(n, 0)//初始化
vector<vector<int>> a(n + 1, vector<int>(m + 1, 0))//二维初始化
c.front()//返回容器中的第一个数据
c.back()//返回容器中的最后一个数据
c.size()//返回实际数据个数 (unsigned类型)
c.begin()//返回首元素的迭代器 (通俗来说就是地址)
c.end()//返回最后一个元素后一个位置的迭代器 (地址)
c.empty()//判断是否为空, 为空返回真, 反之返回假
c.assign(beg, end)//将另外一个容器[x.begin(), x.end()) 里的内容拷贝到c中c.assign(n, val)
c.pop_back()//删除最后一个数据
c.push_back(element)//在尾部加一个数据
c.emplace_back(ele)//在数组中加入一个数据, 和 push_back功能基本一样, 在某些情况下比它效率更高, 支持传入多个构造参数
c.clear()//清除容器中的所有元素
c.insert(pos, x)//向任意迭代器pos插入一个元素x
vector<priority_queue<task, vector<task>, function<bool(const task&, const task&>>> a; //动态加入不同规则优先队列
vector<variant<
    priority_queue<task, vector<task>, TF>,
    priority_queue<task, vector<task>, SF>,
    priority_queue<task, vector<task>, LF>
>> a; //静态加入不同规则优先队列
```

map

```
mp.find(key)//返回键为key的映射的迭代器
mp.erase(it)//删除迭代器对应的键和值
mp.erase(key)//根据映射的键删除键和值
mp.erase(first, last)//删除左闭右开区间迭代器对应的键和值
mp.size()//返回映射的对数
mp.clear()//清空map中的所有元素
mp.insert()//插入元素, 插入时要构造键值对
mp.empty()//如果map为空, 返回true, 否则返回false
mp.begin()//返回指向map第一个元素的迭代器 (地址)
mp.end()//返回指向map尾部的迭代器 (最后一个元素的下一个地址)
mp.rbegin()//返回指向map最后一个元素的迭代器 (地址)
mp.rend()//返回指向map第一个元素前面(上一个)的逆向迭代器 (地址)
mp.count(key)//查看元素是否存在, 因为map中键是唯一的, 所以存在返回1, 不存在返回0
mp.lower_bound(key)//返回一个迭代器, 指向键值>= key的第一个元素
mp.upper_bound(key)//返回一个迭代器, 指向键值> key的第一个元素
//遍历
auto it = mp.begin();
while(it != mp.end()) {
    cout << it->first << " " << it->second << "\n";
    it++;
}
unordered_map: //优点: 内部用哈希表实现, 查找速度非常快 (适用于大量的查询操作)
```

set

```
s.begin()//返回set容器的第一个元素的地址（迭代器）
s.end()//返回set容器的最后一个元素的下一个地址（迭代器）
s.rbegin()//返回逆序迭代器，指向容器元素最后一个位置
s.rend()//返回逆序迭代器，指向容器第一个元素前面的位置
s.clear()//删除set容器中的所有的元素,无返回值
s.empty()//判断set容器是否为空
s.insert(element)//插入一个元素
s.size()//返回当前set容器中的元素个数
erase(iterator)//删除定位器iterator指向的值
erase(first, second) //删除定位器first和second之间的值
erase(key_value)//删除键值key_value的值
s.find(element) //查找set中的某一元素，有则返回该元素对应的迭代器，无则返回结束迭代器
s.count(element)//查找set中的元素出现的个数，由于set中元素唯一，此函数相当于查询element是否出现
s.lower_bound(k)//返回大于等于k的第一个元素的迭代器
s.upper_bound(k)//返回大于k的第一个元素的迭代器
set<int> s1; // 默认从小到大排序
set<int, greater<int> > s2; // 从大到小排序
struct Point {
    int x, y;
    bool operator < (const Point &p) const {
        // 按照点的横坐标从小到大排序,如果横坐标相同,纵坐标从小到大
        if(x == p.x)
            return y < p.y;
        return x < p.x;
    }
};
set<Point> s;
multiset //元素可以重复，且元素有序
unordered_set//元素无序且只能出现一次
unordered_multiset//元素无序可以出现多次
```

离散化

```
//数组
sort(b + 1, b + n + 1);
int len = unique(b + 1, b + n + 1) - (b + 1); // 去重后长度
for (int i = 1; i <= n; i++)
{
    int pos = lower_bound(b + 1, b + len + 1, a[i]) - b;
    a[i] = pos; // 这里 pos 就是压缩后的值的下标
}
//vector
sort(b.begin(), b.end());
b.erase(unique(b.begin(), b.end()), b.end());
for (int i = 0; i < n; ++i)
{
    a[i] = lower_bound(b.begin(), b.end(), a[i]) - b.begin() + 1;
}
```

pbds

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef long long ll;
typedef std::pair<ll,int> pii; // 把 key 放 first, 插入 id 放 second
// 默认比较器: 按 key 排序 (first), 相等时按 id (second)
struct DefaultCmp {
    bool operator()(const pii& a,const pii& b) const {
        if(a.first!=b.first) return a.first<b.first;
        return a.second<b.second;
    }
};
// 平衡树 SBT 模板类, 可以传不同比较器
template<class Cmp=DefaultCmp>
struct SBT {
    tree<pii,null_type,Cmp,rb_tree_tag,tree_order_statistics_node_update> tr;
    //tree<int, null_type, std::less<int>, rb_tree_tag,
    tree_order_statistics_node_update> tr;
    //当set用
    int cnt=0;
    void clear(){tr.clear();cnt=0;}
    void ins(ll k){tr.insert({k,++cnt});} // 插入值 k
    void del(ll k){tr.erase(tr.lower_bound({k,0}));} // 删除值 k
    int rnk(ll k){return tr.order_of_key({k,0})+1;} // 查询排名(1-based)
    ll kth(int x){return tr.find_by_order(x-1)->first;} // 查询第x小
    ll pre(ll k){return kth(rnk(k)-1);} // 查询前驱
    ll nxt(ll k){return kth(rnk(k)+1);} // 查询后继
};
```

字符串

字符串哈希

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int MAXN = 1e6+10;
ll h1[MAXN], h2[MAXN], p1[MAXN], p2[MAXN];
ll b1 = 257, b2 = 263;
const ll mod1 = 1e9+7, mod2 = 1e9+9;
int n;
string s;
void init_hash()
{
    p1[0] = p2[0] = 1;
    h1[0] = h2[0] = 0;
    for (int i = 1; i <= n; i++)
    {
        ll v = s[i-1] - 'a' + 1;
        h1[i] = (h1[i-1] * b1 + v) % mod1;
        h2[i] = (h2[i-1] * b2 + v) % mod2;
    }
}
```



```

        p1[i] = p1[i-1] * b1 % mod1;
        p2[i] = p2[i-1] * b2 % mod2;
    }
}
ll get_hash(int l, int r, int flag)
{
    if (l > r) return 0;
    if (flag > 0) return (h1[r] - h1[l-1] * p1[r-l+1] % mod1 + mod1) % mod1;
    else return (h2[r] - h2[l-1] * p2[r-l+1] % mod2 + mod2) % mod2;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n >> s;
    init_hash();
    ll ha = get_hash(1, n, 1);
    ll hb = get_hash(1, n, 0);
    cout << "Hash_mod1 = " << ha << ", Hash_mod2 = " << hb <<
    return 0;
}

```

KMP

```

void pre()//A=" "+A
{
    p[1]=0;
    int j=0;
    for(int i=1;i<m;i++)
    {
        while(j>0&&B[j+1]!=B[i+1]) j=p[j];
        if(B[j+1]==B[i+1]) j++;
        p[i+1]=j;
    }
}

int kmp()
{
    int ans=0,j=0;
    for(int i=0;i<n;i++)
    {
        while(j>0&&B[j+1]!=A[i+1]) j=p[j];
        if(B[j+1]==A[i+1])j++;
        if(j==m)
        {
            ans++;
            j=0;
        }
    }
    return ans;
}

```

- KMP自动机

定义 $lst_{i,j}$ 表示第 i 个前缀的所有border中，最长的使得下一位刚好是 j 的位置，具体规则如下：当 $s_{i+1} = j$ 时， $lst_{i,j} = i$ ；当 $s_{i+1} \neq j$ 时， $lst_{i,j} = lst_{p_i,j}$ 。

```
void pre()
{
    int j=0;
    if(n>1) dp[1][(int)(s[2]-'a')]=1;
    for(int i=2;i<=n;i++)
    {
        while(j>0&&s[j+1]!=s[i]) j=nxt[j];
        if(s[j+1]==s[i]) j++;
        nxt[i]=j;
        for(int k=0;k<26;k++)
        {
            if(i<n&&s[i+1]==(char)('a'+k)) dp[i][k]=i;
            else dp[i][k]=dp[nxt[i]][k];
        }
    }
}

signed main()
{
    scanf("%s", (s+1));
    scanf("%d", &q);
    n=strlen(s+1);
    pre();
    while(q--)
    {
        scanf("%s", (s+1+n));
        int len=n+strlen(s+1+n);
        int j=nxt[n];
        for(int i=n+1;i<=len;i++)
        {
            while(j>0&&s[j+1]!=s[i])
            {
                if(j<=n)
                {
                    j=dp[j][(int)(s[i]-'a')];
                }
                else j=nxt[j];
            }
            if(s[j+1]==s[i]) j++;
            nxt[i]=j;
            printf("%d ",nxt[i]);
        }
        printf("\n");
    }
}
```

Trie字典树

- 前缀查询

```
bool insert(char *s)
{
    int len=strlen(s);
```

```

int u=1;
bool flag=false;
for(int i=0;i<len;i++)
{
    int c=s[i]-'0';
    if(!ch[u][c])
        ch[u][c]=++tot;
    else if(i==len-1)
    {
        flag=true;
    }
    u=ch[u][c];
    if(bo[u]) flag=true;
}
bo[u]=true;
return flag;
}

```

- 最长异或和

```

void insert(int a)
{
    int u=0;
    for(int i=31;i>=0;i--)
    {
        int x=(a>>i)&1;
        if(!ch[u][x])
        {
            ch[u][x]=++num;
        }
        u=ch[u][x];
    }
}

int find(int a)
{
    int u=0,res=0;
    for(int i=31;i>=0;i--)
    {
        int x=(a>>i)&1;
        if(ch[u][!x])
        {
            res=res*2+!x;
            u=ch[u][!x];
        }
        else
        {
            res=res*2+x;
            u=ch[u][x];
        }
    }
    res=res^a;
    return res;
}

```

AC自动机

- 给你一个文本串 S 和 n 个模式串 $T_1 \sim T_n$, 请你分别求出每个模式串 T_i 在 S 中出现的次数。

```
#include<bits/stdc++.h>
#define maxn 2000001
using namespace std;
char s[maxn],T[maxn];
int n,cnt,vis[200051],ans,in[maxn],Map[maxn];
struct kkk{
    int son[26],fail,flag,ans;
    void clear(){memset(son,0,sizeof(son)),fail=flag=ans=0;}
}trie[maxn];
queue<int>q;
void insert(char* s,int num){
    int u=1,len=strlen(s);
    for(int i=0;i<len;i++){
        int v=s[i]-'a';
        if(!trie[u].son[v])trie[u].son[v]=++cnt;
        u=trie[u].son[v];
    }
    if(!trie[u].flag)trie[u].flag=num;
    Map[num]=trie[u].flag;
}
void getFail(){
    for(int i=0;i<26;i++)trie[0].son[i]=1;
    q.push(1);
    while(!q.empty()){
        int u=q.front();q.pop();
        int Fail=trie[u].fail;
        for(int i=0;i<26;i++){
            int v=trie[u].son[i];
            if(!v){trie[u].son[i]=trie[Fail].son[i];continue;}
            trie[v].fail=trie[Fail].son[i]; in[trie[v].fail]++;
            q.push(v);
        }
    }
}
void topu(){
    for(int i=1;i<=cnt;i++){
        if(in[i]==0)q.push(i);
        while(!q.empty()){
            int u=q.front();q.pop();vis[trie[u].flag]=trie[u].ans;
            int v=trie[u].fail;in[v]--;
            trie[v].ans+=trie[u].ans;
            if(in[v]==0)q.push(v);
        }
    }
}
void query(char* s){
    int u=1,len=strlen(s);
    for(int i=0;i<len;i++)
        u=trie[u].son[s[i]-'a'],trie[u].ans++;
}
int main(){
    scanf("%d",&n); cnt=1;
```

```

for(int i=1;i<=n;i++){
    scanf("%s",s);
    insert(s,i);
}getFail();scanf("%s",T);
query(T);topu();
for(int i=1;i<=n;i++)printf("%d\n",vis[Map[i]]);
}

```

数论

快速幂

```

long long kpow(long long a,long long b)
{
    long long res=1;
    while(b)
    {
        if(b&1) res=res*a%MOD;
        a=a*a%N;
        b>>=1;
    }
    return res;
}

```

组合数学

- 排列组合

```

const long long MOD=1e9+7;
const int N=1e7+10;
long long fac[N+10], inv[N+10];
long long kpow(long long a, long long b, long long p)
{
    long long res = 1;
    while(b>0)
    {
        if (b%2) res=res*a%p;
        a=a*a%p;
        b/=2;
    }
    return res;
}
void pre()
{
    fac[0] =inv[0]=1;
    for (int i=1;i<=N;i++)
    {
        fac[i]=fac[i-1]*i%MOD;
    }
    inv[N]=kpow(fac[N],MOD-2,MOD);
    for (int i=N-1;i>0;i--)
    {
        inv[i] = inv[i+1]*(i+1)%MOD;
    }
}

```

```

}
long long C(int n, int k)
{
    if (k>n) return 0;
    return fac[n]*inv[k]%MOD*inv[n-k]%MOD;
}
long long A(int n,int k)
{
    if(k>n) return 0;
    return fac[n]*inv[n-k]%MOD;
}

```

- 第二类斯特林数

```

long long S(int n, int k)
{
    if (k > n) return 0;
    if (n == 0 && k == 0) return 1;
    long long res = 0;
    for (int j = 0; j <= k; ++j)
    {
        long long term = C(k, j) * kpow(j, n, MOD) % MOD;
        if((k - j) & 1) res = (res - term) % MOD;
        else res = (res + term) % MOD;
    }
    if (res < 0) res += MOD;
    res = res * inv[k] % MOD;
    return res;
}

```

质数

- Eratosthenes筛选法

```

for(int i=2;i<=N;i++)
{
    if(!vis[i])
    {
        prime[++tot]=i;
        for(int j=i;j<=N/i;j++)
        {
            vis[i*j]=1;
        }
    }
}

```

- 线性筛

```

void init_sieve() {
    memset(v, 0, sizeof(v));
    m = 0;
    for (int i = 2; i <= N; i++) {
        if (v[i] == 0) { // i 是质数
            v[i] = i;
            prime[++m] = i;
        }
        for (int j = 1; j <= m && prime[j] <= v[i] && prime[j] <= N/i; j++) {
            v[prime[j] * i] = prime[j];
        }
    }
}

```

- 质因数分解(试除法)

```

void divide(int n)
{
    tot=0;
    for(int i=2;i*i<=n;i++)
    {
        if(n%i==0)
        {
            p[++tot]=i;
            c[tot]=0;
            while(n%i==0)
            {
                n/=i;
                c[tot]++;
            }
        }
    }
    if(n>1)
    {
        p[++tot]=n; c[tot]=1;
    }
}

```

- 质因数分解(预处理)

```

void init_sieve()
{
    for (int i=2;i<=N;i++)
    {
        if (spf[i]==0)
        {
            spf[i]=i;
            for(int j=i;j<=N/i;j++)
            {
                if(spf[i*j]==0) spf[i*j]=i;
            }
        }
    }
}

```

```

void divide(int n)
{
    tot = 0;
    while (n > 1) {
        int pr = spf[n];
        p[++tot] = pr;
        c[tot] = 0;
        while (n % pr == 0) {
            n /= pr;
            c[tot]++;
        }
    }
}

```

- $1 - n$ 每个数的因子的预处理

```

void init()
{
    for(int i = 1; i < N; i++)
        for(int j = i; j < N; j += i)
            d[j].push_back(i);
}

```

线性求逆元

```

inv[0]=inv[1]=1;
for (int i=2;i<=n;i++) inv[i]=(long long)(mo-mo/i)*inv[mo%i]%mo;

```

欧拉函数

- 即对任意满足 $\gcd(a, b) = 1$ 的整数 a, b , 有 $\varphi(ab) = \varphi(a)\varphi(b)$ 。
- 特别地, 当 n 是奇数时 $\varphi(2n) = \varphi(n)$ 。
- $n = \sum_{d|n} \varphi(d)$ 。
- 若 $n = p^k$, 其中 p 是质数, 那么 $\varphi(n) = p^k - p^{k-1}$ 。(根据定义可知)
- 由唯一分解定理, 设 $n = \prod_{i=1}^s p_i^{k_i}$, 其中 p_i 是质数, 有 $\varphi(n) = n \times \prod_{i=1}^s \frac{p_i-1}{p_i}$ 。
- 欧拉反演

$$\sum_{i=1}^n \gcd(i, n) = \sum_d \sum_{i=1}^n [d | i][d | n] \varphi(d) = \sum_d \left\lfloor \frac{n}{d} \right\rfloor [d | n] \varphi(d) = \sum_{d|n} \left\lfloor \frac{n}{d} \right\rfloor \varphi(d)$$

$$\sum_{i=1}^n \sum_{j=1}^n \gcd(i, j) = \sum_{i=1}^n \sum_{j=1}^n \sum_{\substack{d|i \\ d|j}} \varphi(d) = \sum_{d=1}^n \varphi(d) \cdot \sum_{i=1}^n [d | i] \cdot \sum_{j=1}^n [d | j] = \sum_{d=1}^n \left\lfloor \frac{n}{d} \right\rfloor^2 \cdot \varphi(d)$$

- 求一个数的欧拉函数


```
int euler_phi(int n) {
    int ans = n;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) {
            ans = ans / i * (i - 1);
            while (n % i == 0) n /= i;
        }
    if (n > 1) ans = ans / n * (n - 1);
    return ans;
}
```

- 多个数的欧拉函数

```
vector<int> pri;
bool not_prime[N];
int phi[N];
void pre(int n) {
    phi[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!not_prime[i]) {
            pri.push_back(i);
            phi[i] = i - 1;
        }
        for (int pri_j : pri) {
            if (i * pri_j > n) break;
            not_prime[i * pri_j] = true;
            if (i % pri_j == 0) {
                phi[i * pri_j] = phi[i] * pri_j;
                break;
            }
            phi[i * pri_j] = phi[i] * phi[pri_j];
        }
    }
}
```

同余

- 同余方程求解

```
void exgcd(ll a, ll b, ll &gcd, ll &x, ll &y)
{
    if (b == 0)
    {
        gcd = a; x = 1; y = 0;
        return ;
    }
    exgcd(b, a % b, gcd, x, y);
    ll t = x;
    x = y;
    y = t - a / b * y;
}
//ax+by=n
//a*x ≡ n (mod b)
if(n%gcd!=0) cout<<"NO"<<"\n";
```

```
cout<<((x*n/gcd)%abs(b/gcd)+abs(b/gcd))%abs(b/gcd)<<'\n';
```

- 中国剩余定理

$m_1, m_2 \cdots m_n$ 两两互质时成立。

```
int IntChina(int n)
{
    int Mi,x0,y0,d,Ans=0;
    int M=1;
    for(int i=1;i<=n;i++) M*=m[i];
    for(int i=1;i<=n;i++)
    {
        Mi=M/m[i];
        Exgcd(Mi,m[i],d,x0,y0);
        Ans=(Ans+Mi*x0*a[i])%M;
    }
    return (Ans+M)%M;
}
```

- 拓展欧几里得解同余方程

$m_1, m_2 \cdots m_n$ 两两不互质。

```
for(int i=1;i<n;i++)
{
    cin>>a>>b;
    b=(b-ans%a+a)%a;
    exgcd(lcm,a,d,x,y);

    if(b%d!=0)
    {
        flag=1;
    }
    else
    {
        ans+=((x*(b/d)%a+a)%a)*lcm;
        lcm=lcm/d*a;
        ans=(ans%lcm+lcm)%lcm;
    }
}
```

- 线性求逆元

```
inv[1] = 1;
for (int i = 2; i <= n; ++i) {
    inv[i] = (long long)(p - p / i) * inv[p % i] % p;
}
```

博弈论

- Nim游戏:

$a_1 \wedge a_2 \wedge \dots \wedge a_n \neq 0$, 先手必胜。

- 有向图游戏

```
void dfs()
{
    while(!q.empty())
    {
        int x=q.front();
        q.pop();
        temp=0;
        memset(vis,0,sizeof(vis));
        for(int i=head[x];i;i=edge[i].next)
        {
            int y=edge[i].to;
            temp=max(temp,sg[y]);
            vis[sg[y]]=1;
        }
        for(int i=0;i<=temp+1;i++)
        {
            if(!vis[i])
            {
                sg[x]=i;
                break;
            }
        }
        for(int i=rehead[x];i;i=reedge[i].next)
        {
            int y=reedge[i].to;
            out[y]--;
            if(!out[y])
            {
                q.push(y);
            }
        }
    }
}
```

FFT(多项式乘法)

```
#include<iostream>
#include<cstdio>
#include<cmath>
using namespace std;
const int MAXN=1e7+10;
inline int read()
{
    char c=getchar();int x=0,f=1;
    while(c<'0' || c>'9'){if(c=='-')f=-1;c=getchar();}
    while(c>='0'&&c<='9'){x=x*10+c-'0';c=getchar();}
    return x*f;
}
```

```

}
const double Pi=acos(-1.0);
struct complex
{
    double x,y;
    complex (double xx=0,double yy=0){x=xx,y=yy;}
}a[MAXN],b[MAXN];
complex operator + (complex a,complex b){ return complex(a.x+b.x , a.y+b.y);}
complex operator - (complex a,complex b){ return complex(a.x-b.x , a.y-b.y);}
complex operator * (complex a,complex b){ return complex(a.x*b.x-a.y*b.y ,
a.x*b.y+a.y*b.x);}//不懂的看复数的运算那部分
int N,M;
int l,r[MAXN];
int limit=1;
void fast_fast_tle(complex *A,int type)
{
    for(int i=0;i<limit;i++)
        if(i<r[i]) swap(A[i],A[r[i]]); //求出要迭代的序列
    for(int mid=1;mid<limit;mid<=<1)//待合并区间的中点
    {
        complex wn( cos(Pi/mid) , type*sin(Pi/mid) ); //单位根
        for(int R=mid<<1,j=0;j<limit;j+=R)//R是区间的右端点，j表示前已经到哪个位置了
        {
            complex w(1,0); //幂
            for(int k=0;k<mid;k++,w=w*wn)//枚举左半部分
            {
                complex x=A[j+k],y=w*A[j+mid+k]; //蝴蝶效应
                A[j+k]=x+y;
                A[j+mid+k]=x-y;
            }
        }
    }
}
int main()
{
    int N=read(),M=read();
    for(int i=0;i<=N;i++) a[i].x=read();
    for(int i=0;i<=M;i++) b[i].x=read();
    while(limit<=N+M) limit<=<1,l++;
    for(int i=0;i<limit;i++)
        r[i]= ( r[i>>1]>>1 ) | ( (i&1)<<(l-1) ) ;
    // 在原序列中 i 与 i/2 的关系是：i 可以看做是i/2的二进制上的每一位左移一位得来
    // 那么在反转后的数组中就需要右移一位，同时特殊处理一下复数
    fast_fast_tle(a,1);
    fast_fast_tle(b,1);
    for(int i=0;i<=limit;i++) a[i]=a[i]*b[i];
    fast_fast_tle(a,-1);
    for(int i=0;i<=N+M;i++)
        printf("%d ",(int)(a[i].x/limit+0.5));
    return 0;
}

```

线性代数

- 高斯消元 $O(n^3)$

```
#include<iostream>
#include<algorithm>
#include<cmath>
using namespace std;
const int N=110;
const double eps=1e-8;
double a[N][N]; //存储增广矩阵
int n;
int gauss()
{
    int r,c; //r表示当前要处理的这一行
    for(r=0,c=0;c<n;c++) //遍历每一列
    {
        int t=r;
        for(int i=r;i<n;i++) //找到这一列中元素最大的一行
            if(fabs(a[i][c])>fabs(a[t][c]))
                t=i;
        if(fabs(a[t][c])<eps) continue; //如果元素最大，还是0，那就跳过，去处理下一列
        for(int i=c;i<n;i++) swap(a[t][i],a[r][i]); //把选中的这一行放到“最上面”去
        for(int i=n;i>=c;i--) a[r][i] /=a[r][c]; //把这一行的第c列化成1
        for(int i=r+1;i<n;i++) //把其他行的第c列消成0
            if(fabs(a[i][c])>eps)
            {
                for(int j=n;j>=c;j--)
                    a[i][j]-=a[i][c]*a[r][j];
            }
        r++;
    }
    if(r<n) //如果最后不是严格完全的阶梯型
    {
        for(int i=r;i<n;i++)
            if(fabs(a[i][n])>eps) //0==非零的情况，无解
                return 2;
        return 1; //0==0的情况，有无穷多解
    }
    for(int i=n-1;i>=0;i--) //从下往上的把解给求出来
        for(int j=i+1;j<n;j++)
            a[i][n]-=a[j][n]*a[i][j];
    return 0;
}
int main()
{
    scanf("%d",&n);
    for(int i=0;i<n;i++)
        for(int j=0;j<n+1;j++)
            scanf("%lf",&a[i][j]);
    int t=gauss();
    if(t==2)
        printf("No solution");
    else if(t==1)
        printf("Infinite group solutions");
}
```

```

else
    for(int i=0;i<n;i++)
    {
        if(fabs(a[i][n])<eps)
            a[i][n]=0;
        printf("%.2lf\n",a[i][n]);
    }
    return 0;
}

```

- 高斯消元解异或方程组 $O(n^3)$ (可区分三种解)

```

#include <iostream>
#include <algorithm>
using namespace std;
const int N = 110;
int n;
int a[N][N];
int gauss()
{
    //枚举行列
    int r, c;
    for (r = 0, c = 0; c < n; c++)
    {
        int t = r;
        for (int i = r; i < n; i++)
            if (a[i][c])//找到一个非零数
            {
                t = i;
                break;
            }
        if (!a[t][c]) continue;//如果没找到, 则continue
        //交换两列
        for (int i = c; i <= n; i++) swap(a[t][i], a[r][i]);

        //从 r + 1行开始消零
        for (int i = r + 1; i < n; i++)
            if (a[i][c])//如果这个数不为0则异或
                for (int j = c; j <= n; j++)
                    a[i][j] ^= a[r][j];

        r++;
    }
    //判断是否有解
    if (r < n)
    {
        for (int i = r; i < n; i++)
            if (a[i][n]) return 2;
        return 1;
    }
    //将所有行消掉0
    for (int i = n - 1; i >= 0; i--)
        for (int j = i + 1; j < n; j++)//把每行后面所有系数消掉
            a[i][n] ^= a[i][j] & a[j][n]; //只有1的时候才异或, 所以用&运算
    return 0;
}

```

```

int main()
{
    cin >> n;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n + 1; j++)
            cin >> a[i][j];
    int res = gauss();
    if (res == 0)
    {
        for (int i = 0; i < n; i++) cout << a[i][n] << endl;
    }
    else if (res == 1) puts("Multiple sets of solutions");
    else puts("No solution");
    return 0;
}

```

- 高斯消元法解异或方程组（只能返回唯一解）

```

bitset<1010> matrix[2010]; // matrix[1~n]: 增广矩阵, 0 位置为常数
vector<bool> GaussElimination(
    int n, int m) // n 为未知数个数, m 为方程个数, 返回方程组的解
                // (多解 / 无解返回一个空的 vector)
{
    for (int i = 1; i <= n; i++) {
        int cur = i;
        while (cur <= m && !matrix[cur].test(i)) cur++;
        if (cur > m) return vector<bool>(0);
        if (cur != i) swap(matrix[cur], matrix[i]);
        for (int j = 1; j <= m; j++)
            if (i != j && matrix[j].test(i)) matrix[j] ^= matrix[i];
    }
    vector<bool> ans(n + 1);
    for (int i = 1; i <= n; i++) ans[i] = matrix[i].test(0);
    return ans;
}

```

- 高斯消元（三角形矩阵）

```

//a[i][1]*f[1]+a[i][2]*f[2]+...+a[i][m]*f[m]=a[i][m+1]
for(int i=1;i<m;i++)
{
    double p=a[i+1][i]/a[i][i];
    a[i+1][i]=0;
    a[i+1][i+1]-=a[i][i+1]*p;
    a[i+1][m+1]-=a[i][m+1]*p;
}
f[m]=a[m][m+1]/a[m][m];
for(int i=m-1;i>=1;i--)
f[i]=(a[i][m+1]-a[i][i+1]*f[i+1])/a[i][i];

```

- 矩阵加速

$$\begin{bmatrix} F_{n-1} & F_{n-2} \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} F_n & F_{n-1} \end{bmatrix}$$

```

constexpr int mod = 1000000007;
struct Matrix {
    int a[3][3];
    Matrix() { memset(a, 0, sizeof a); }
    Matrix operator*(const Matrix &b) const {
        Matrix res;
        for (int i = 1; i <= 2; ++i)
            for (int j = 1; j <= 2; ++j)
                for (int k = 1; k <= 2; ++k)
                    res.a[i][j] = (res.a[i][j] + a[i][k] * b.a[k][j]) % mod;
        return res;
    }
} ans, base;
void init() {
    base.a[1][1] = base.a[1][2] = base.a[2][1] = 1;
    ans.a[1][1] = ans.a[1][2] = 1;
}
void qpow(int b) {
    while (b) {
        if (b & 1) ans = ans * base;
        base = base * base;
        b >>= 1;
    }
}
int main() {
    int n = read();
    if (n <= 2) return puts("1"), 0;
    init();
    qpow(n - 2);
    printf("%d\n", ans.a[1][1] % mod);
}

```

动态规划

背包

- 多重背包（二进制优化）

```

const int N=103,M=4e4+3;
int n,m,x,y,z,V,ans,v[N*17],w[N*17],dp[M];
int main(){
    in(m),in(V);
    memset(dp,-127,sizeof(dp));
    dp[0]=0;
    while(m--){
        in(x),in(y),in(z);Re p=1;
        while(z>=p)v[++n]=y*p,w[n]=x*p,z-=p,p<<=1;
        if(z)v[++n]=y*z,w[n]=x*z;
    }
    for(Re i=1;i<=n;++i)
        for(Re j=V;j>=v[i];--j)
            dp[j]=max(dp[j],dp[j-v[i]]+w[i]);
    for(Re i=0;i<=V;++i)ans=max(ans,dp[i]);
    printf("%d\n",ans);
}

```



```
}
```

- 混合背包

```
#include<algorithm>
#include<cstdio>
using namespace std;
int x,a,b,c,d,T,n,i,j,t,f[1010],v[100010],w[100010];
int main(){
    scanf("%d:%d%d:%d%d",&a,&b,&c,&d,&n);
    T=c*60+d-a*60-b;
    for(i=1;i<=n;i++){
        scanf("%d%d%d",&a,&b,&c);
        if(!c){
            for(j=a;j<=T;j++){
                f[j]=max(f[j],f[j-a]+b);
            }
            continue;
        }
        int p=1;
        while(c>=p){v[++t]=p*a,w[t]=p*b,c-=p,p<<=1;}
        v[++t]=c*a,w[t]=c*b;
    }
    for(i=1;i<=t;i++){
        for(j=T;j>=v[i];j--){
            f[j]=max(f[j],f[j-v[i]]+w[i]);
        }
    }
    printf("%d",f[T]);
    return 0;
}
```

- 二维费用

```
#include<algorithm>
#include<cstdio>
using namespace std;
int tmp,T1,T2,x,y,n,i,j,k,v1[105],v2[105],w[105],dp[105][105],ans[105][105];
int main(){
    scanf("%d",&n);
    for(i=1;i<=n;i++)scanf("%d%d%d",&v1[i],&v2[i],&w[i]);
    scanf("%d%d",&T1,&T2);
    for(i=1;i<=n;i++){
        for(j=T1;j>=v1[i];j--){
            for(k=T2;k>=v2[i];k--){
                if(dp[j][k]<(tmp=dp[j-v1[i]][k-v2[i]]+w[i])){
                    dp[j][k]=tmp;
                    ans[j][k]=ans[j-v1[i]][k-v2[i]]+w[i];
                }
                else if(dp[j][k]==tmp)ans[j][k]=min(ans[j][k],ans[j-v1[i]][k-v2[i]]+w[i]);
            }
        }
    }
    printf("%d",ans[T1][T2]);
}
```

- 分组背包

```
#include<bits/stdc++.h>
```

```

using namespace std;
int a,i,j,k,T,n,m,f[1010],w[1010],v[1010],id[105][1010];
int main(){
    scanf("%d%d",&T,&n);
    for(i=1;i<=n;i++){
        scanf("%d%d%d",&v[i],&w[i],&a);
        id[a][++id[a][0]]=i;m=max(m,a);
    }
    for(k=1;k<=m;k++){
        for(j=T;j>=0;j--){
            for(i=1;i<=id[k][0];i++){
                a=id[k][i];
                if(j>=v[a])f[j]=max(f[j-v[a]]+w[a],f[j]);
            }
        }
        printf("%d",f[T]);
        return 0;
    }
}

```

- 依赖背包

```

#include<algorithm>
#include<cstdio>
using namespace std;
int T,n,i,j,a[65],s,t,r1,r2,r[65][3],v[65],w[65],f[32010];
int main(){
    scanf("%d%d",&T,&n);
    v[0]=0xffffffff;
    for(i=1;i<=n;i++){
        scanf("%d%d%d",&v[i],&s,&a[i]);
        if(a[i])r[a[i]][++r[a[i]][0]]=i;w[i]=v[i]*s;
    }
    for(i=1;i<=n;i++){
        for(j=T;j>=v[i]&&(!a[i]);j--){
            t=j-v[i],r1=r[i][1],r2=r[i][2];
            f[j]=max(f[j],f[t]+w[i]);
            if(v[r1]<=t)f[j]=max(f[j],f[t-v[r1]]+w[i]+w[r1]);
            if(v[r2]<=t)f[j]=max(f[j],f[t-v[r2]]+w[i]+w[r2]);
            if(v[r1]+v[r2]<=t)f[j]=max(f[j],f[t-v[r1]-v[r2]]+w[i]+w[r1]+w[r2]);
        }
        printf("%d",f[T]);
    }
}

```

最长公共子序列

```

for(int i=1;i<=n;i++)
{
    cin>>a[i];
    belong[a[i]]=i;
}
for(int i=1;i<=n;i++) cin>>b[i];
int len=0;
for(int i=1;i<=n;i++)
{
    if(belong[b[i]]>f[len]) f[++len]=belong[b[i]];
}

```

```

else
{
    int k=upper_bound(f+1,f+len+1,belong[b[i]])-f;
    f[k]=min(f[k],belong[b[i]]);
}
}
cout<<len<<'\n';

```

最长上升子序列

```

//ans中存储的是对应长度LIS的最小末尾元素
for(auto &x:c)
{
    int pos=lower_bound(ans.begin(),ans.end(),x)-ans.begin();
    if(pos==ans.size()) ans.emplace_back(x);
    else ans[pos]=x;
    //pre[i]=index[pos-1] 需要记录每个元素
}

```

区间dp

```

for(int i=1;i<=2*n;i++) dpmin[i][i]=0;
for(int len=1;len<=n;len++)
for(int i=1;i<=2*n-1;i++)
{
    int j=len+i-1;
    for(int k=i;k<j&& k<=2*n-1;k++)
    {
        dpmin[i][j]=min(dpmin[i][j],dpmin[i][k]+dpmin[k+1][j]+sum[j]-sum[i-1]);
        dpmax[i][j]=max(dpmax[i][j],dpmax[i][k]+dpmax[k+1][j]+sum[j]-sum[i-1]);
    }
}

```

树形dp

- 基础dp

```

void dfs(int x)
{
    dp[x][1]=r[x];
    for(int i=head[x];i;i=edge[i].nxt)
    {
        int y=edge[i].to;
        dfs(y);
        dp[x][0]+=max(dp[y][1],dp[y][0]);
        dp[x][1]+=dp[y][0];
    }
}

```

- 树形背包

```

int dfs(int u,int fa)

```

```

{
    if(u>=n-m+1&&u<=n)
    {
        dp[u][1]=money[u];
        return 1;
    }
    int size=0,p=0;
    for(int i=head[u];i;i=edge[i].nxt)
    {
        int v=edge[i].to;
        int w=edge[i].w;
        if(v==fa) continue;
        p=dfs(v,u);
        size+=p;
        for(int j=size;j>=1;j--)
        {
            for(int k=1;k<=j&&k<=p;k++)
            {
                dp[u][j]=max(dp[u][j],dp[u][j-k]+dp[v][k]-w);
            }
        }
    }
    return size;
}

```

- 换根

```

void dfs(int u,int fa)
{
    dep[u]=dep[fa]+1;
    siz[u]=1;
    f[1]+=(ll)dep[u];
    for(int i=head[u];i;i=edge[i].nxt)
    {
        int v=edge[i].to;
        if(v==fa) continue;
        dfs(v,u);
        siz[u]+=siz[v];
    }
}

void dfs2(int u,int fa)
{
    for(int i=head[u];i;i=edge[i].nxt)
    {
        int v=edge[i].to;
        if(v==fa) continue;
        f[v]=f[u]+(ll)n-2LL*siz[v];
        dfs2(v,u);
    }
}

dfs(1,0);
dfs2(1,0);

```

状压dp

```
void deal_first()
{
    for(int i=0;i<(1<<n);i++)
    {
        if(i&(i<<1)) continue;
        int x=0;
        for(int j=0;j<n;j++)
        {
            if(i&(1<<j)) x++;
        }
        s[++cnt]=i;
        num[cnt]=x;
    }
}

void dp()
{
    f[0][1][0]=1;
    for(int i=1;i<=n;i++)
    for(int j=1;j<=cnt;j++)
    for(int l=0;l<=k;l++)
    {
        if(l>=num[j])
        {
            for(int j1=1;j1<=cnt;j1++)
            {
                if(!(s[j]&s[j1])&&!(s[j]&(s[j1]<<1))&&!(s[j]&(s[j1]>>1))))
                {
                    f[i][j][l]+=f[i-1][j1][l-num[j]];
                }
            }
        }
    }

    for(int i=1;i<=cnt;i++) ans+=f[n][i][k];
    printf("%lld",ans);
}
```

数位dp

- 递推版

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
ll a,b;
ll cnta[20],cntb[20];
ll dp[20],ten[20],num[20];
void solve(ll x,ll cnt[])
{

```

```

for(int i=0;i<20;i++) num[i]=0;
int len=0;
while(x)
{
    num[++len]=x%10;
    x/=10;
}
for(int i=len;i>=1;i--)
{
    for(int j=0;j<=9;j++) cnt[j]+=dp[i-1]*num[i];
    for(int j=0;j<num[i];j++) cnt[j]+=ten[i-1];
    ll num2=0;
    for(int j=i-1;j>=1;j--) num2=num2*10+num[j];
    cnt[num[i]]+=num2+1;
    cnt[0]-=ten[i-1];
}
}
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin>>a>>b;
    ten[0]=1;
    for(int i=1;i<=15;i++)
    {
        dp[i]=dp[i-1]*10+ten[i-1];
        //这两部分前一个是来自前 i-1 位数字的贡献，后一个是来自第 i 位的数字的贡献
        ten[i]=ten[i-1]*10;
    }
    solve(a-1,cnta);
    solve(b,cntb);
    for(int i=0;i<=9;i++) cout<<cntb[i]-cnta[i]<<' ';
    cout<<'\n';
    return 0;
}

```

- dfs版

```

#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
int len,num[20];
ll a,b;
ll dp[15][10][2]; //dp[i][j][k]表示搜到第i位，前一位是j，的k状态方案totnum;
ll dfs(int pos,int pre,int st,int limit)
    //pos当前位置,pre前一位数,st判断前面是否全是0,limit最高位限制
{
    if(pos>len) return 1;
    if(dp[pos][pre][limit]!=-1) return dp[pos][pre][limit]; //没有最高位限制，已经搜过了
    ll ret=0;
    int res=limit?num[len-pos+1]:9; //当前位最大数字
    for(int i=0;i<=res;i++)
    {

```

```

        if(abs(i-pre)<2) continue;
        if(st&&i==0) ret+=dfs(pos+1,-2,1,limit&&(i==res)); //如果仍然是全前导0状态，下一位随意
        else ret+=dfs(pos+1,i,0,limit&&(i==res));
    }
    if(!st) dp[pos][pre][limit]=ret; //没有前导0时记录结果
    return ret;
}
11 solve(11 x)
{
    len=0;
    while(x) num[++len]=x%10,x/=10;
    memset(dp,-1,sizeof(dp));
    return dfs(1,-2,1,1);
}
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin>>a>>b;
    cout<<slove(b)-slove(a-1)<<'\n';
    return 0;
}

```

概率dp

- 求概率

```

cin>>w>>b;
for(int i=1;i<=w;i++) dp[i][0]=1.0;
for(int i=1;i<=w;i++)
{
    for(int j=1;j<=b;j++)
    {
        dp[i][j]=double(i)/(i+j);
        if(j>=3) dp[i][j]+=double(j)/(i+j)*double(j-1)/(i+j-1)*
            double(j-2)/(i+j-2)*dp[i][j-3];
        if(i>1&&j>=2) dp[i][j]+=double(j)/(i+j)*double(j-1)/(i+j-1)*
            double(i)/(i+j-2)*dp[i-1][j-2];
    }
}
cout<<fixed<<setprecision(9)<<dp[w][b]<<'\n';

```

- 求期望

```

cin>>n;
for(int i=1;i<=n;i++)
{
    cin>>x;
    dp[i]=dp[i-1]+x*(1+3*len2+3*len);
    len2=(len2+2*len+1)*x;
    len=x*(len+1);
}
cout<<fixed<<setprecision(1)<<dp[n]<<'\n';

```

- 有后效性期望

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N=1010;
int n,m,stx,sty;
double a[N][N],f[N];
void solve(int x)
{
    for (int i=1;i<=m;i++)
        for (int j=1;j<=m+1;j++)
            a[i][j] = 0;
    //a[j][1]*f[i][1]+a[j][2]*f[i][2]+...+a[j][m]*f[i][m]=a[j][m+1]
    //表示从第j列移动形成的方程
    for(int j=1;j<=m;j++)
    {
        if(j==1)
        {
            a[j][1]=2;
            a[j][2]=-1;
            a[j][m+1]=3+f[1];
        }
        else if(j==m)
        {
            a[j][m]=2;
            a[j][m-1]=-1;
            a[j][m+1]=3+f[m];
        }
        else
        {
            a[j][j]=3;
            a[j][j+1]=-1;
            a[j][j-1]=-1;
            a[j][m+1]=4+f[j];
        }
    }
    //高斯消元（三对角矩阵）
    for(int i=1;i<=m;i++)
    {
        double p=a[i+1][i]/a[i][i];
        a[i+1][i]=0;
        a[i+1][i+1]-=a[i][i+1]*p;
        a[i+1][m+1]-=a[i][m+1]*p;
    }
    f[m]=a[m][m+1]/a[m][m];
    for(int i=m-1;i>=1;i--)
        f[i]=(a[i][m+1]-a[i][i+1]*f[i+1])/a[i][i];
}
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin>>n>>m;
    cin>>stx>>sty;
```



```

    if(m==1)
    {
        cout<<fixed<<setprecision(10)<<2.0*(n-stx)<<'\n';
        return 0;
    }
    for(int i=n-1;i>=stx;i--) solve(i);
    cout<<fixed<<setprecision(10)<<f[sty]<<'\n';
    return 0;
}

```

dp优化

- 斜率优化
- 求min维护下凸包 (k单调递增) , 求max维护上凸包 (k单调递减)
- $\frac{Y(j_2)-Y(j_1)}{X(j_2)-X(j_1)} \leq 2S[i]$ 维护下凸包, 反之维护上凸包。
- 移项要遵循的原则是: 把含有 $function[i] * function[j]$ 的表达式看作斜率 k_0 乘以未知数 x , 含有 $dp[i]$ 的项必须要在 $b*b$ 的表达式中, 含有 $function[j]$ 的项必须在 y 的表达式中。如果未知数 x 的表达式单调递减, 最好让等式两边同乘个 -1 , 使其变为单增。

```

11 n,L,h=1,t=0,q[N],dp[N],sum[N];
11 X(11 j) {return sum[j];}
11 Y(11 j) {return dp[j]+(sum[j]+L)*(sum[j]+L);}
long double slope(11 i,11 j)
{
    return (long double)(Y(j)-Y(i))/(X(j)-X(i));
}
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin>>n>>L;L++;
    for(int i=1;i<=n;i++)
    {
        cin>>sum[i];
        sum[i]+=sum[i-1]+1;
    }
    q[++t]=0;
    for(int i=1;i<=n;i++)
    {
        while(h<t&&slope(q[h],q[h+1])<=2*sum[i]) h++;
        //若第一个元素小于等于k0那么出队
        //至少要有两个元素 h<t。出队判断时尽量加上等号
        int j=q[h];
        dp[i]=dp[j]+(sum[i]-sum[j]-L)*(sum[i]-sum[j]-L);
        while(h<t&&slope(q[t-1],q[t])>=slope(q[t],i)) t--;
        //新加入的点导致k1>k2, 所以删除中间点。
        //至少要有两个元素 h<t。入队判断时尽量加上等号
        q[++t]=i;
    }
    cout<<dp[n]<<"\n";
    return 0;
}

```

图论

最短路

- Floyd

```
for(int k=1;k<=n;k++)
for(int i=1;i<=n;i++)
for(int j=1;j<=n;j++)
{
    if(i!=j&&i!=k&&j!=k) f[i][j]=min(f[i][j],f[i][k]+f[k][j]);
}
```

- 01 BFS （边权只有0或1的最短路问题）

```
vector<int> dis(n + 1, INF);
deque<int> q;
dis[s] = 0;
q.push_front(s);
while (!q.empty()) {
    int u = q.front();
    q.pop_front();
    for (auto [v, w] : g[u]) {
        if (dis[v] > dis[u] + w) {
            dis[v] = dis[u] + w;
            if (w == 0) q.push_front(v);
            else q.push_back(v);
        }
    }
}
```

- Dijkstra

```
void dijkstra(int from)
{
    memset(dis,0x3f3f3f3f,sizeof(dis));
    memset(vis,0,sizeof(vis));
    dis[from]=0;
    q.push(make_pair(0,from));
    while(!q.empty())
    {
        int u=q.top().second;
        q.pop();
        if(vis[u]) continue;
        vis[u]=1;
        for(int i=head[u];i;i=edge[i].next)
        {
            int v=edge[i].to;
            if(!vis[v]&&dis[v]>dis[u]+edge[i].dis)
            {
                dis[v]=dis[u]+edge[i].dis;
                q.push(make_pair(dis[v],v));
            }
        }
    }
}
```

```

    }

}

```

- SPFA

```

void spfa(int s)
{
    memset(dis, -INF, sizeof(dis));
    dis[s] = 0;
    q.push(s);
    vis[s] = 1;
    while(q.size())
    {
        int x = q.front();
        q.pop();
        vis[x] = 0;
        for(int i = head[x]; i; i = edge[i].next)
        {
            int y = edge[i].to;
            if(dis[y] > dis[x] + edge[i].dis && !vis[y])
            {
                dis[y] = dis[x] + edge[i].dis;
                if(!vis[y])
                {
                    q.push(y);
                    vis[y] = 1;
                    //cnt[y]++; //判负环
                    //if(cnt[y] > n) return true; // 存在负环
                }
            }
        }
    }
}

```

- 多源最短路

```

void multi_source_bfs(int n, vector<int>& sources) {
    queue<int> q;
    fill(dist + 1, dist + n + 1, INF);
    for (int s : sources) {
        dist[s] = 0;
        q.push(s);
    }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) {
            if (dist[v] == INF) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}

```

```
}
```

Tarjan缩点

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N=1e4+10,M=1e5+10;
int n,m,vis[N],du[N],dp[N],dfn[N],low[N],step,w[N],cnt,sccw[N],id[N];
vector<int> g1[N], g2[N]; // 原图和缩点后的图

stack<int> s;
queue<int> q;

void dfs(int u)
{
    s.push(u);
    vis[u]=1;
    dfn[u]=low[u]=++step;
    for(auto v:g1[u])
    {
        if(!dfn[v])
        {
            dfs(v);
            low[u]=min(low[u],low[v]);
        }
        else if(vis[v])
        {
            low[u]=min(low[u],dfn[v]);
        }
    }
    if(dfn[u]==low[u])
    {
        cnt++;
        int t;
        do
        {
            t=s.top(); s.pop();
            vis[t]=0;
            sccw[cnt]+=w[t];
            id[t]=cnt;
        }while(t!=u);
    }
}

void tarjan()
{
    for(int i=1;i<=n;i++) if(!dfn[i]) dfs(i);
    for(int u=1;u<=n;u++)
    {
        for(auto v:g1[u])
        {
            int x=id[u], y=id[v];
            if(x!=y)
            {

```

```

        g2[x].push_back(y);
        du[y]++;
    }
}
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin>>n>>m;
    for(int i=1;i<=n;i++) cin>>w[i];
    int u,v;
    for(int i=1;i<=m;i++)
    {
        cin>>u>>v;
        g1[u].push_back(v);
    }
    tarjan();
    for(int i=1;i<=cnt;i++)
    {
        dp[i]=sccw[i];
        if(!du[i]) q.push(i);
    }
    while(!q.empty())
    {
        int t=q.front(); q.pop();
        for(auto v:g2[t])
        {
            dp[v]=max(dp[v],dp[t]+sccw[v]);
            du[v]--;
            if(!du[v]) q.push(v);
        }
    }
    int ans=0;
    for(int i=1;i<=cnt;i++) ans=max(ans,dp[i]);
    cout<<ans<<'\n';
    return 0;
}

```

最小生成树

- kruskal

```

int find(int x)
{
    if(fa[x]!=x) return fa[x]=find(fa[x]);
    else return x;
}

void unionn(int r1,int r2)
{
    int r3=find(r1);
    int r4=find(r2);
    if(r3!=r4)

```

```

    {
        fa[r3]=r4;
    }
}
for(int i=1;i<=tot;i++)
{
    int xx=find(edge[i].x);
    int yy=find(edge[i].y);
    if(xx!=yy)
    {
        unionn(xx,yy);
        ans[++num]=edge[i].dis;
    }
}

```

- 严格次小生成树

```

void dfs(int u,int father)
{
    f[u][0]=father;
    dep[u]=dep[father]+1;
    for(int i=1;i<=20;i++)
    {
        if(dep[u]<(1<<i)) break;
        f[u][i]=f[f[u][i-1]][i-1];
        ll a=max1[u][i-1],b=max1[f[u][i-1]][i-1];
        ll c=max2[u][i-1],d=max2[f[u][i-1]][i-1];
        max1[u][i]=max(a,b);
        if(a==b) max2[u][i]=max(c,d);
        else if(a>b) max2[u][i]=max(c,b);
        else if(b>a) max2[u][i]=max(d,a);
    }
    for(int i=head[u];i;i=edge1[i].nxt)
    {
        int v=edge1[i].to;
        if(v==father) continue;
        max1[v][0]=edge1[i].w;
        dfs(v,u);
    }
}

void change(int x,int y,ll w)
{
    ll maxx=0;
    for(int i=20;i>=0;i--)
    {
        if(dep[f[x][i]]>=dep[y])
        {
            if(max1[x][i]<w) maxx=max(maxx,max1[x][i]);
            else if(max1[x][i]==w&&max2[x][i]!=0) maxx=max(maxx,max2[x][i]);
            x=f[x][i];
        }
    }
    if(maxx>0) ans1=min(ans1,ans-maxx+w);
}

void solve()

```

```

{
    for(int i=1;i<=tot;i++)
    {
        if(edge[i].vis) continue;
        int u=edge[i].from,v=edge[i].to;
        int la=lca(u,v);
        change(u,la,edge[i].w);
        change(v,la,edge[i].w);
    }
    cout<<ans1<<'\\n';
}

```

LCA

```

void deal(int u,int father)
{
    dep[u]=dep[father]+1;
    for(int i=0;i<=14;i++)
        f[u][i+1]=f[f[u][i]][i];
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v==father) continue;
        f[v][0]=u;
        dis[v]=dis[u]+edge[i].dis;
        deal(v,u);
    }
}

int lca(int x,int y)
{
    if(dep[x]<dep[y]) swap(x,y);
    for(int i=15;i>=0;i--)
    {
        if(dep[f[x][i]]>=dep[y])
        {
            x=f[x][i];
        }
        if(x==y) return x;
    }
    for(int i=15;i>=0;i--)
    {
        if(f[x][i]!=f[y][i])
        {
            x=f[x][i];
            y=f[y][i];
        }
    }
    return f[x][0];
}

int dist(int x,int y)
{
    return dis[x]+dis[y]-2*dis[lca(x,y)];
}

```

```
}
```

树的直径

- 任意一点出发，最远点一定是某条直径的端点
- 直径路径是“最长简单路径”
- 树的“中心”在直径上
- 所有最长路径都会经过直径中点
- 两遍dfs

```
function<void(int, int)> dfs = [&](int u, int f)
{
    fa[u] = f;
    for (auto v:g[u])
    {
        if (v == f) continue;
        dep[v] = dep[u] + 1;
        if (dep[v] > dep[e]) e=v;
        dfs(v, u);
    }
};
int e=1;
dfs(1,0);
for(int i=1;i<=n;i++) dep[i]=0;
int s=e;
dfs(e,0);
vector<int> path;
for(int x=c;x=f[x])
{
    path.push_back(x);
    if(x==s) break;
}
```

- 树形dp（存在负权边的情况下求解出树的直径）

```
void dfs(int u, int fa) {
    d1[u] = d2[u] = 0;
    for (int v : E[u]) {
        if (v == fa) continue;
        dfs(v, u);
        int t = d1[v] + 1;
        if (t > d1[u])
            d2[u] = d1[u], d1[u] = t;
        else if (t > d2[u])
            d2[u] = t;
    }
    d = max(d, d1[u] + d2[u]);
}

int main() {
    scanf("%d", &n);
    for (int i = 1; i < n; i++) {
        int u, v;
```



```

scanf("%d %d", &u, &v);
E[u].push_back(v), E[v].push_back(u);
}
dfs(1, 0);
printf("%d\n", d);
return 0;
}

```

树的中心

在树中，如果节点 x 作为根节点时，从 x 出发的最长链最短，那么称 x 为这棵树的中心。

- 树的中心不一定唯一，但最多有两个，且这两个中心是相邻的。
- 树的中心一定位于树的直径上。
- 树上所有点到其最远点的路径一定交会于树的中心。
- 当树的中心为根节点时，其到达直径端点的两条链分别为最长链和次长链。
- 当通过在两棵树间连一条边以合并为一棵树时，连接两棵树的中心可以使新树的直径最小。
- 树的中心到其他任意节点的距离不超过树直径的一半。

//d1是x子树内的最长链，d2是不与d1重叠的最长链，up是x子树外的最长链。

```

void dfsd(int cur, int fa) {
    for (int i = head[cur]; i; i = edges[i].nxt) {
        int nxt = edges[i].to, w = edges[i].val;
        if (nxt == fa) continue;
        dfsd(nxt, cur);
        if (d1[nxt] + w > d1[cur]) {
            d2[cur] = d1[cur];
            d1[cur] = d1[nxt] + w;
        } else if (d1[nxt] + w > d2[cur]) {
            d2[cur] = d1[nxt] + w;
        }
    }
}

```

```

void dfsu(int cur, int fa) {
    for (int i = head[cur]; i; i = edges[i].nxt) {
        int nxt = edges[i].to, w = edges[i].val;
        if (nxt == fa) continue;
        up[nxt] = up[cur] + w;
        if (d1[nxt] + w != d1[cur]) {
            up[nxt] = max(up[nxt], d1[cur] + w);
        } else {
            up[nxt] = max(up[nxt], d2[cur] + w);
        }
        dfsu(nxt, cur);
    }
}

```

```

void GetTreeCenter() {
    dfsd(1, 0);
    dfsu(1, 0);
    for (int i = 1; i <= n; ++i) {
        int maxLen = max(d1[i], up[i]);
        if (maxLen < mini) {

```

```

        mini = maxLen;
        x = i;
        y = 0;
    } else if (maxLen == mini) {
        y = i;
    }
}
}
}

```

树的重心

如果在树中选择某个节点并删除，这棵树将分为若干棵子树，统计子树节点数并记录最大值。取遍树上所有节点，使此最大值取到最小的节点被称为整个树的重心。

- 树的重心如果不唯一，则至多有两个，且这两个重心相邻。
- 以树的重心为根时，所有子树的大小都不超过整棵树大小的一半。
- 树中所有点到某个点的距离和中，到重心的距离和是最小的；如果有两个重心，那么到它们的距离和一样。
- 把两棵树通过一条边相连得到一棵新的树，那么新的树的重心在连接原来两棵树的重心的路径上。
- 在一棵树上添加或删除一个叶子，那么它的重心最多只移动一条边的距离。

```

const int N = 50005;
int n, siz[N];
long long dp[N], ans[N];
vector<int> g[N], centroids;
// 求 1 号节点到所有其他节点的距离和
void dfs1(int u, int fa) {
    siz[u] = 1;
    dp[u] = 0;
    for (int v : g[u]) {
        if (v == fa) continue;
        dfs1(v, u);
        siz[u] += siz[v];
        dp[u] += dp[v] + siz[v]; // 子树节点到 u 的距离和
    }
}
// 通过换根 DP 求所有节点为树根时对应的距离和
void dfs2(int u, int fa) {
    for (int v : g[u]) {
        if (v == fa) continue;
        ans[v] = ans[u] - siz[v] + (n - siz[v]);
        dfs2(v, u);
    }
}
// 求树的重心
void get_centroids() {
    dfs1(1, 0);
    ans[1] = dp[1];
    dfs2(1, 0);
    long long mini = std::numeric_limits<long long>::max();
    for (int i = 1; i <= n; i++) {
        if (ans[i] < mini) {
            mini = ans[i];
        }
    }
}

```

```

        centroids = {i};
    } else if (ans[i] == mini)
        centroids.push_back(i);
    }
}

```

数据结构

树状数组

```

//x必须大于0
int lowbit(int x) { return x&(-x); }
void add(int x,int y){ for(;x<=n;x+=lowbit(x)) sum[x]+=y; }
int qsum(int x)
{
    int res=0;
    for(;x>=1;x-=lowbit(x)) res+=sum[x];
    return res;
}
auto find_kth=[&](int k)
{
    int pos = 0;
    for(int i = 20; i >= 0; i--)
    {
        int nxt = pos + (1 << i);
        if(nxt <= m && sum[nxt] < k)
        {
            k -= sum[nxt];
            pos = nxt;
        }
    }
    return pos + 1;
};

```

- 二维

```

void update(int x,int y,int z)
{
    int j=x;
    while(j<=n)
    {
        int i=y;
        while(i<=m)
        {
            c[j][i]+=z;
            i+=lowbit(i);
        }
        j+=lowbit(j);
    }
}

ll sum(int x,int y)
{
    ll res=0;

```

```

int j=x;
while(j>0)
{
    int i=y;
    while(i>0)
    {
        res+=c[j][i];
        i-=lowbit(i);
    }
    j-=lowbit(j);
}
return res;
}

```

ST表

```

Log[0]=-1;
for(int i=1;i<=n;i++) Log[i]=Log[i>>1]+1,f[i][0]=a[i];
for(int i=1;i<=18;i++)
{
    for(int j=1;j+(1<<i)-1<=n;j++)
    {
        f[j][i]=max(f[j][i-1],f[j+(1<<i)-1][i-1]);
    }
}
int l,r;
while(m--)
{
    cin>>l>>r;
    int x=Log[r-l+1];
    int maxx=max(f[l][x],f[r-(1<<x)+1][x]);
    cout<<maxx<<'\\n';
}

```

线段树

- 单点修改

```

void modify(int k,int l,int r,int x,int y)
{
    if(x<l||x>r) return ;
    if(l==x&&r==x)
    {
        sum[k]+=y;
        return;
    }
    int mid=(l+r)/2;
    modify(k<<1,l,mid,x,y);
    modify(k<<1|1,mid+1,r,x,y);
    sum[k]=sum[k<<1]+sum[k<<1|1];
}
long long query(int k,int l,int r,int x,int y)
{

```

```

    if(y<l||x>r) return 0;
    if(l>=x&& r<=y) return sum[k];
    int mid=(l+r)/2;
    long long res=0;
    res=query(k<<1,l,mid,x,y);
    res+=query(k<<1|1,mid+1,r,x,y);
    return res;
}

```

- 区间修改

```

long long sum[N*4],addsum[N*4];
void build(int k,int l,int r)
{
    if(l==r)
    {
        sum[k]=a[l];
        return ;
    }
    int mid=(l+r)/2;
    build(k<<1,l,mid);
    build(k<<1|1,mid+1,r);
    sum[k]=sum[k<<1]+sum[k<<1|1];
}

void Add(int k,int l,int r,int v)
{
    addsum[k]+=v;
    sum[k]+=(long long)v*(r-l+1);
}

void pushdown(int k,int l,int r,int mid)
{
    if(addsum[k]==0) return;
    Add(k<<1,l,mid,addsum[k]);
    Add(k<<1|1,mid+1,r,addsum[k]);
    addsum[k]=0;
}

long long query(int k,int l,int r,int x,int y)
{
    if(l>=x&& r<=y) return sum[k];
    int mid=(l+r)/2;
    long long res=0;
    pushdown(k,l,r,mid);
    if(x<=mid)res+=query(k<<1,l,mid,x,y);
    if(y>mid)res+=query(k<<1|1,mid+1,r,x,y);
    return res;
}

void modify(int k,int l,int r,int x,int y,int v)
{
    if(l>=x&& r<=y)
        return Add(k,l,r,v);
    int mid=(l+r)/2;
    pushdown(k,l,r,mid);
}

```

```

        if(x<=mid) modify(k<<1,l,mid,x,y,v);
        if(mid<y) modify(k<<1|1,mid+1,r,x,y,v);
        sum[k]=sum[k<<1]+sum[k<<1|1];
    }
}

```

树链抛分

```

#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N=1e5+10;
struct Edge
{
    int to,nxt;
}edge[N<<1];
int head[N],tot;
int dep[N],fa[N],siz[N],son[N],top[N];
int dfn[N],rev[N],sum[N<<2],tag[N<<2];
int n,m,w[N],root,p;
void addedge(int from,int to)
{
    edge[++tot].to=to;
    edge[tot].nxt=head[from];
    head[from]=tot;
}
void dfs1(int u)
{
    siz[u]=1;
    for(int i=head[u];i;i=edge[i].nxt)
    {
        int v=edge[i].to;
        if(v==fa[u]) continue;
        fa[v]=u;dep[v]=dep[u]+1;
        dfs1(v);
        if(siz[son[u]]<siz[v]) son[u]=v;
        siz[u]+=siz[v];
    }
}
void dfs2(int u)
{
    if(!son[u]) return;
    dfn[son[u]]=++dfn[0];rev[dfn[0]]=son[u];
    top[son[u]]=top[u];
    dfs2(son[u]);
    for(int i=head[u];i;i=edge[i].nxt)
    {
        int v=edge[i].to;
        if(v==fa[u]||v==son[u]) continue;
        dfn[v]=++dfn[0];rev[dfn[0]]=v;
        top[v]=v;
        dfs2(v);
    }
}
void Add(int k,int l,int r,int v)

```

```

{
    tag[k]=(tag[k]+v)%p;
    sum[k]=(sum[k]+1LL*(r-l+1)*v%p)%p;
}
void pushdown(int k,int l,int r)
{
    if(tag[k]==0) return;
    int mid=(l+r)>>1;
    Add(k<<1,l,mid,tag[k]);
    Add(k<<1|1,mid+1,r,tag[k]);
    tag[k]=0;
}
void build(int k,int l,int r)
{
    if(l==r)
    {
        sum[k]=w[rev[l]];
        return;
    }
    int mid=(l+r)>>1;
    build(k<<1,l,mid);
    build(k<<1|1,mid+1,r);
    sum[k]=(sum[k<<1]+sum[k<<1|1])%p;
}
void change(int k,int l,int r,int x,int y,int v)
{
    if(x<=l&&r<=y) return Add(k,l,r,v);
    pushdown(k,l,r);
    int mid=(l+r)>>1;
    if(x<=mid) change(k<<1,l,mid,x,y,v);
    if(y>mid) change(k<<1|1,mid+1,r,x,y,v);
    sum[k]=(sum[k<<1]+sum[k<<1|1])%p;
}
void modify(int x,int y,int v)
{
    int fx=top[x],fy=top[y];
    while(fx!=fy)
    {
        if(dep[fx]<dep[fy]) swap(fx,fy),swap(x,y);
        change(1,1,n,dfn[fx],dfn[x],v);
        x=fa[fx],fx=top[x];
    }
    if(dep[x]<dep[y]) swap(x,y);
    change(1,1,n,dfn[y],dfn[x],v);
    return ;
}
int asksum(int k,int l,int r,int x,int y)
{
    if(x<=l&&r<=y) return sum[k];
    int mid=(l+r)>>1,ans=0;
    pushdown(k,l,r);
    if(x<=mid) ans=(ans+asksum(k<<1,l,mid,x,y))%p;
    if(y>mid) ans=(ans+asksum(k<<1|1,mid+1,r,x,y))%p;
    return ans;
}
int qsum(int x,int y)

```

```

{
    int fx=top[x],fy=top[y],ans=0;
    while(fx!=fy)
    {
        if(dep[fx]<dep[fy]) swap(fx,fy),swap(x,y);
        ans=(ans+asksum(1,1,n,dfn[fx],dfn[x]))%p;
        x=fa[fx],fx=top[x];
    }
    if(dep[x]<dep[y]) swap(x,y);
    ans=(ans+asksum(1,1,n,dfn[y],dfn[x]))%p;
    return ans;
}
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin>>n>>m>>root>>p;
    for(int i=1;i<=n;i++) cin>>w[i],w[i]%=p;
    int op,z,x,y;
    for(int i=1;i<n;i++)
    {
        cin>>x>>y;
        addedge(x,y);
        addedge(y,x);
    }
    dep[root]=dfn[0]=dfn[root]=1,top[root]=rev[1]=root;
    dfs1(root);dfs2(root);build(1,1,n);
    while(m--)
    {
        cin>>op;
        if(op==1) cin>>x>>y>>z,modify(x,y,z);
        else if(op==2) cin>>x>>y,cout<<qsum(x,y)%p<<'\\n';
        else if(op==3) cin>>x>>z,change(1,1,n,dfn[x],dfn[x]+siz[x]-1,z);
        //更改x为根的子树
        else cin>>x,cout<<asksum(1,1,n,dfn[x],dfn[x]+siz[x]-1)%p<<'\\n';
    }

    return 0;
}

```

笛卡尔树

```

s.push(1),curbot=1;
for(int i=2;i<=n;i++)
{
    while(!s.empty()&&a[s.top()]>a[i]) s.pop();
    if(s.empty())
    {
        ls[i]=curbot;
        curbot=i;
    }
    else
    {
        ls[i]=rs[s.top()];
        rs[s.top()]=i;
    }
}

```



```
    }  
    s.push(i);  
}
```

杂项

分块

```
//建立  
num = sqrt(n);  
for (int i = 1; i <= num; i++)  
    st[i] = n / num * (i - 1) + 1, ed[i] = n / num * i;  
ed[num] = n;  
for (int i = 1; i <= num; i++) {  
    for (int j = st[i]; j <= ed[i]; j++) {  
        belong[j] = i;  
    }  
    size[i] = ed[i] - st[i] + 1;  
}  
  
//更改  
void Sort(int k) {  
    for (int i = st[k]; i <= ed[k]; i++) t[i] = a[i];  
    sort(t + st[k], t + ed[k] + 1);  
}  
  
void PushDown(int x) {  
    if (delta[x] != 0x3f3f3f3f3f3f3f11) // 用该值标记块内没有被整体赋值  
        for (int i = st[x]; i <= ed[x]; i++) a[i] = t[i] = delta[x];  
    delta[x] = 0x3f3f3f3f3f3f3f11;  
}  
  
void Modify(int l, int r, int c) {  
    int x = belong[l], y = belong[r];  
    PushDown(x);  
    if (x == y) {  
        for (int i = l; i <= r; i++) a[i] = c;  
        Sort(x);  
        return;  
    }  
    PushDown(y);  
    for (int i = l; i <= ed[x]; i++) a[i] = c;  
    for (int i = st[y]; i <= r; i++) a[i] = c;  
    Sort(x);  
    Sort(y);  
    for (int i = x + 1; i < y; i++) delta[i] = c;  
}  
  
int Binary_Search(int l, int r, int c) {  
    int ans = l - 1, mid;  
    while (l <= r) {  
        mid = (l + r) / 2;  
        if (t[mid] <= c)  
            ans = mid, l = mid + 1;  
    }  
}
```

```

        else
            r = mid - 1;
    }
    return ans;
}

int Answer(int l, int r, int c) {
    int ans = 0, x = belong[l], y = belong[r];
    PushDown(x);
    if (x == y) {
        for (int i = l; i <= r; i++)
            if (a[i] <= c) ans++;
        return ans;
    }
    PushDown(y);
    for (int i = l; i <= ed[x]; i++)
        if (a[i] <= c) ans++;
    for (int i = st[y]; i <= r; i++)
        if (a[i] <= c) ans++;
    for (int i = x + 1; i <= y - 1; i++) {
        if (0x3f3f3f3f3f3f3f3f == delta[i])
            ans += Binary_Search(st[i], ed[i], c) - st[i] + 1;
        else if (delta[i] <= c)
            ans += size[i];
    }
    return ans;
}

```

莫队算法

- 普通莫队
- 从序列的第一个询问开始计算答案，第一个询问通过直接暴力算出，复杂度为 $O(n)$ ，后面的询问在前一个询问的基础上得到答案。

```

bool cmp(node x, node y)
{
    if (id[x.l] == id[y.l])
    {
        if (id[x.l] & 1 == 1) return x.r < y.r;
        else return x.r > y.r;
    }
    else return id[x.l] < id[y.l];
}

int m, n, len, l, r;
ll cnt[N];
void add(int x)
{
    cnt[x]++;
    if (cnt[x] > 1) now.x = now.x + cnt[x] * (cnt[x] - 1) - (cnt[x] - 1) * (cnt[x] - 2);
}
void pop(int x)
{
    cnt[x]--;
    if (cnt[x] > 0) now.x = now.x + cnt[x] * (cnt[x] - 1) - (cnt[x] + 1) * cnt[x];
}

```

```

}
void divgcd(ll x,ll y,int id)
{
    if(!x) x=0,y=1;
    else
    {
        int g=__gcd(x,y);
        x/=g,y/=g;
    }
    ans[id].x=x,ans[id].y=y;
}
int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin>>n>>m;
    len=sqrt(n);
    for(int i=1;i<=n;i++)
    {
        cin>>a[i];
        id[i]=(i-1)/len+1;
    }
    for(int i=1;i<=m;i++)
    {
        cin>>qs[i].l>>qs[i].r;
        qs[i].id=i;
    }
    sort(qs+1,qs+m+1,cmp);
    for(int i=qs[1].l;i<=qs[1].r;i++)
    {
        add(a[i]);
    }
    now.y=(qs[1].r-qs[1].l+1)*(qs[1].r-qs[1].l);
    divgcd(now.x,now.y,qs[1].id);
    l=qs[1].l,r=qs[1].r;
    for(int i=2;i<=m;i++)
    {
        while(l<qs[i].l) pop(a[l++]);
        while(l>qs[i].l) add(a[--l]);
        while(r<qs[i].r) add(a[++r]);
        while(r>qs[i].r) pop(a[r--]);
        now.y=(qs[i].r-qs[i].l+1)*(qs[i].r-qs[i].l);
        divgcd(now.x,now.y,qs[i].id);
    }
    for(int i=1;i<=m;i++) cout<<ans[i].x<<"/"<<ans[i].y<<endl;
    return 0;
}

```

- 带修莫队
- 我们可以强行让它可以修改，就像 DP 一样，可以强行加上一维 **时间维**，表示这次操作的时间。时间维表示经历的修改次数。即把询问 $[l, r]$ 变成 $[l, r, time]$ 。

```

long long qsize;
struct query {

```

```

long long id, t, l, r;
bool operator<(query b) const {
    if (l / qsize != b.l / qsize) {
        return l / qsize > b.l / qsize;
    } else if (r / qsize != b.r / qsize) {
        return r / qsize > b.r / qsize;
    } else {
        return t > b.t;
    }
}
} q[150009];
struct operation {
    long long p, x;
} r[150009];
char op;
long long n, m, x, y, cur, qcnt, rcnt, mp[1500009], a[150009], ans[150009];
void add(long long x) {
    if (!mp[x]) {
        cur += 1;
    }
    mp[x] += 1;
}
void del(long long x) {
    mp[x] -= 1;
    if (!mp[x]) {
        cur -= 1;
    }
}
void process() {
    sort(q + 1, q + qcnt + 1);
    long long L = 1, R = 0, last = 0;
    for (long long i = 1; i <= qcnt; i++) {
        while (R < q[i].r) {
            add(a[++R]);
        }
        while (R > q[i].r) {
            del(a[R--]);
        }
        while (L > q[i].l) {
            add(a[--L]);
        }
        while (L < q[i].l) {
            del(a[L++]);
        }
        while (last < q[i].t) {
            last += 1;
            if (r[last].p >= L && r[last].p <= R) {
                add(r[last].x);
                del(a[r[last].p]);
            }
            swap(a[r[last].p], r[last].x);
        }
        while (last > q[i].t) {
            if (r[last].p >= L && r[last].p <= R) {
                add(r[last].x);
                del(a[r[last].p]);
            }
        }
    }
}

```

```

    }
    swap(a[r[last].p], r[last].x);
    last -= 1;
}
ans[q[i].id] = cur;
}
}
signed main() {
    cin.tie(nullptr);
    ios::sync_with_stdio(false);
    cin >> n >> m;
    qsize = pow(n, 2.0 / 3.0);
    for (long long i = 1; i <= n; i++) {
        cin >> a[i];
    }
    for (long long i = 1; i <= m; i++) {
        cin >> op >> x >> y;
        if (op == 'Q') {
            ++qcnt, q[qcnt] = {qcnt, rcnt, x, y};
        } else if (op == 'R') {
            r[++rcnt] = {x, y};
        }
    }
    process();
    for (long long i = 1; i <= qcnt; i++) {
        cout << ans[i] << '\n';
    }
}

```

分词

```

vector<string> split(const string &s, char c)
{
    vector<string> res;
    string cur;
    istringstream ss(s);
    while (getline(ss, cur, c)) {
        res.push_back(cur);
    }
    return res;
}
vector<string> fms=split(s, '=');

```